



Inżynieria Oprogramowania

dr Katarzyna Grzesiak-Kopeć





03

Inżynieria wymagań



Co to jest wymaganie?

Wymaganie w inżynierii jest pojedynczą, udokumentowaną potrzebą określonego produktu czy usługi, albo sposobu ich działania.

Jest to stwierdzenie identyfikujące potrzebne cechy, możliwości, charakterystyki lub jakość systemu, aby był on wartościowy i pożyteczny dla użytkownika



Jakie wymagania?



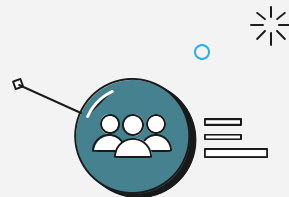
Funkcjonalne: możliwość złożenia zamówienia;
generowanie wypłaty



Pozafunkcjonalne: minimum 5 zamówień na
minutę; 10 000 MTBF; maksymalnie 8h na
przeszkolenie użytkownika



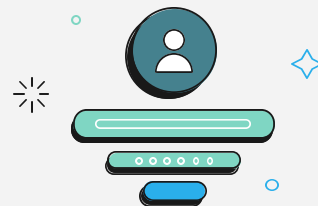
Ograniczenia: czujnik odporny na wilgoć; istniejący
system fakturowania



Spisanie wymagań to sztuka!

Nie ma uniwersalnego sposobu

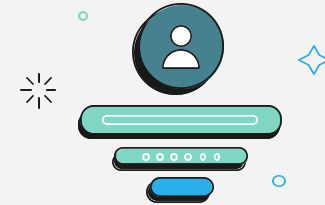
- ✓ Ucz się na cudzych doświadczeniach
- ✓ Korzystaj ze sprawdzonych praktyk



SRS

**IEEE Recommended Practice for
Software Requirements
Specifications**

IEEE Std 830-1998
(Revision of
IEEE Std 830-1993)



Software Requirement Specification

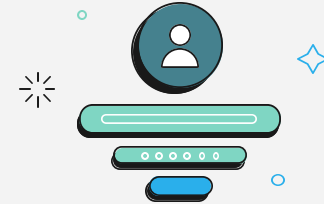
- ✓ Specyfikacja konkretnego produktu oprogramowania realizującego zadaną funkcjonalność w określonym środowisku
- ✓ Może być napisana przez jedną lub wiele osób reprezentujących dostawcę albo/i klienta



SRS zyski

Software Requirement Specification

- ✓ Specyfikacja konkretnego produktu oprogramowania realizującego zadaną funkcjonalność w określonym środowisku
- ✓ Może być napisana przez jedną lub wiele osób reprezentujących dostawcę albo/i klienta



SRS zyski



Uzgodnienie co ma produkt robić



Zmniejsza wysiłek związany z budową systemu



Podstawa szacowania kosztów i harmonogramu



Podstawa walidacji i weryfikacji



Podstawa dla rozbudowy systemu



Ułatwia przenoszenie: nowi użytkownicy, nowe maszyny, nowy klient



SRS zakres



Funkcjonalność: co robi?



Zewnętrzne interfejsy: jak komunikuje się?



Wydajność



Atrybuty: przenośność, poprawność, bezpieczeństwo etc.



Ograniczenia projektowe: standardy, języki programowania etc.



Dobra specyfikacja



01 Poprawna

02 Jednoznaczna

03 Kompletna

04 Spójna

05 Ustalająca priorytety
ważności i stabilności

06 Weryfikowalna

07 Umożliwiająca
śledzenie powiązań

08 Modyfikowalna

SRS

IEEE Recommended Practice for Software Requirements Specifications

IEEE Std 830-1998
(Revision of
IEEE Std 830-1993)

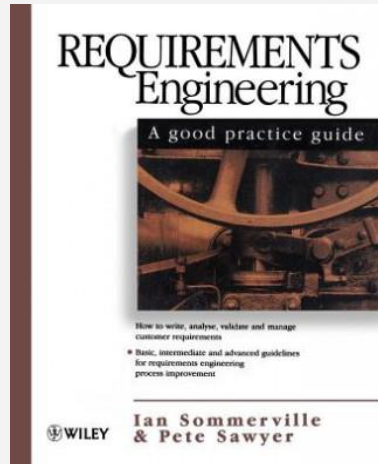
Table of Contents

1. Introduction
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 Definitions, acronyms, and abbreviations
 - 1.4 References
 - 1.5 Overview
 2. Overall description
 - 2.1 Product perspective
 - 2.2 Product functions
 - 2.3 User characteristics
 - 2.4 Constraints
 - 2.5 Assumptions and dependencies
 3. Specific requirements (See 5.3.1 through 5.3.8 for explanations of possible specific requirements. See also Annex A for several different ways of organizing this section of the SRS.)
- Appendixes
- Index

Model Sommerville'a - Sawyera

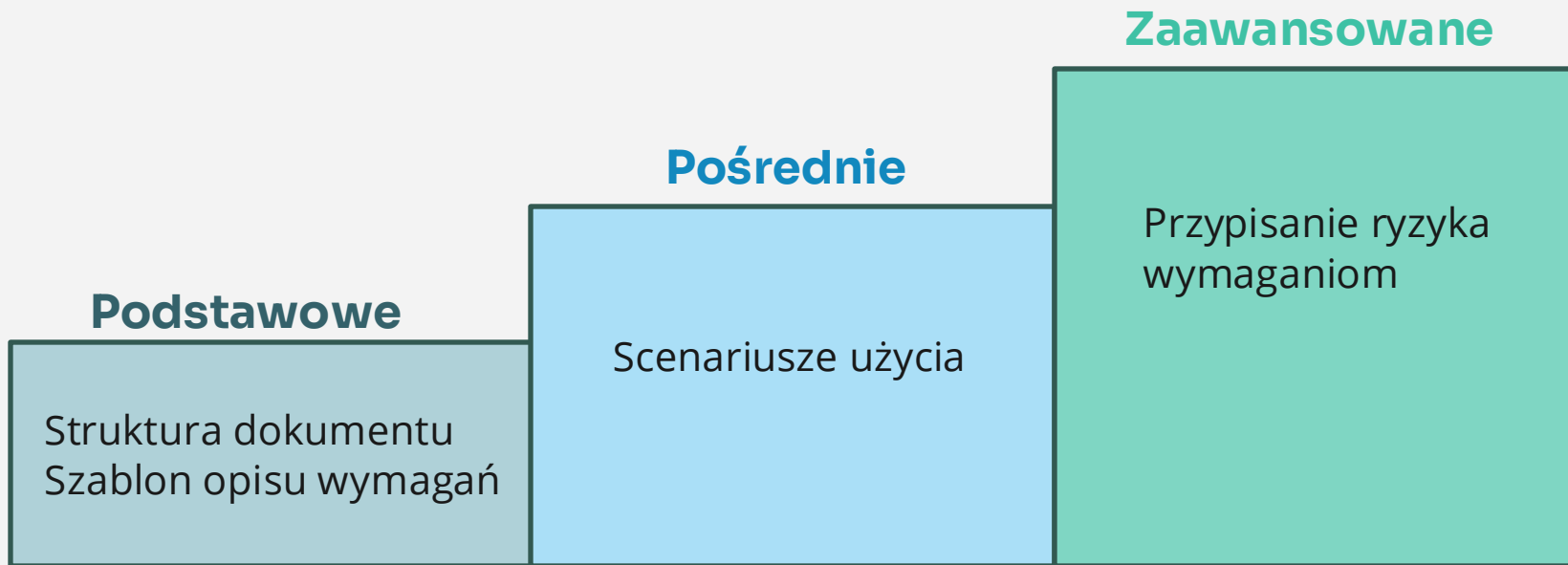
Ian Sommerville & Pete Sawyer

Requirements Engineering: A Good Practice Guide, Wiley, 1997



www.youtube.com/@IanSommerville

Podział dobrych praktyk



Podział dobrych praktyk SRS

Podstawowe

Pośrednie

Zaawansowane

1. Zdefiniuj standardową strukturę dokumentu
2. Wyjaśnij, jak korzystać z dokumentu
3. Dołącz streszczenie wymagań
4. Opracuj uzasadnienie biznesowe dla systemu
5. Zdefiniuj terminy specjalistyczne
6. Wybierz czytelny szablon dokumentu
7. Pomóż czytelnikom znaleźć informację
8. Uczyń dokument łatwym do zmiany

Podział dobrych praktyk SRS

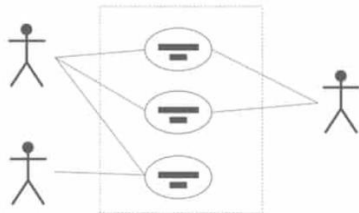
Podstawowe

Pośrednie

Zaawansowane

1.	Zdefiniuj standardy		Podst. 36	Pośr. 21	Zaaw. 9
2.	Wyjaśnij, jak korzysta	Dokument SRS	8	—	—
3.	Dołącz streszczenie	Zbieranie wymagań	6	6	1
4.	Opracuj uzasadnienie	Analiza i negocjacja wymagań	5	2	1
5.	Zdefiniuj terminy	Opisywanie wymagań	4	1	—
6.	Wybierz czytelny s	Modelowanie systemu	3	3	—
7.	Pomóż czytelnikom	Walidacja wymagań	4	3	1
7.	Pomóż czytelnikom	Zarządzanie wymaganiami	4	3	2
8.	Uczyń dokument	IW dla systemów krytycznych	2	3	4

Use Case



**Robert C. Martin
(Uncle Bob)**

Author of Clean Code and many influential books, Co-author of the Agile Manifesto, **USA**

Przypadki użycia

It shouldn't take longer than 15 minutes to teach someone how to write a use case!

Robert Martin

Przypadki użycia

UC 1 Loguj

Aktor:	Użytkownik
Stan systemu przed:	System uruchomiony
Stan systemu po:	Użytkownik otrzymał dostęp do odpowiednich zasobów

SCENARIUSZ PODSTAWOWY

- 1 Użytkownik wyraża chęć zalogowania się.
- 2 System prosi o wprowadzenie identyfikatora oraz hasła.
- 3 Użytkownik wprowadza żądane informacje i zatwierdza je.
- 4 System stwierdza, że wprowadzono odpowiednie dane, weryfikuje tożsamość użytkownika oraz dokonuje autoryzacji.
- 5 System daje użytkownikowi dostęp do odpowiadających mu zasobów.

Przypadki użycia

UC 1 Loguj

Aktor:	Użytkownik
Stan systemu przed:	System uruchomiony
Stan systemu po:	Użytkownik otrzymał dostęp do odpowiednich zasobów

SCENARIUSZ PODSTAWOWY

- 1 Użytkownik wyraża chęć zalogowania się.
- 2 System prosi o wprowadzenie identyfikatora oraz hasła.
- 3 Użytkownik wprowadza żądane informacje i zatwierdza je.
- 4 System stwierdza, że wprowadzono odpowiednie dane, weryfikuje tożsamość użytkownika oraz dokonuje autoryzacji.
- 5 System daje użytkownikowi dostęp do odpowiadających mu zasobów.

Tutaj może być scenariusz alternatywny.

Przypadki użycia

UC 1 Loguj

Aktor:	Użytkownik
Stan systemu przed:	System uruchomiony
Stan systemu po:	Użytkownik otrzymał dostęp do odpowiednich zasobów

SCENARIUSZ PODSTAWOWY

- 1 Użytkownik wyraża chęć zalogowania się.
- 2 System prosi o wprowadzenie identyfikatora oraz hasła

3 Użytkownik wprowadza

4 System stwierdza
tożsamość użytkownika

5 System daje użytkownikowi

SCENARIUSZE ALTERNATYWNE

3a Rezygnacja

3a1 Użytkownik rezygnuje z akcji logowania, co kończy PU.

3b Anulowanie

3b1 Użytkownik anuluje wprowadzone informacje.

3b2 Powrót do pkt. 2.

Przypa

SCENARIUSZE ALTERNATYWNE	
UC 1 Lo	4a Brak lub błędne dane
	4a1 System stwierdza, że brakuje pewnych informacji lub, że są one niepoprawne.
	4a2 System informuje użytkownika o wykrytych problemach i powrót do pkt. 2.
Stan sy	4b Błąd uwierzytelnienia
Star	4b1 System stwierdza, że w systemie nie figuruje albo użytkownik o podanym identyfikatorze, albo para identyfikator-hasło .
SCENAR	4b2 System informuje użytkownika o wykrytych problemach i powrót do pkt. 2.
1 Użytk	4c Brak autoryzacji
2 Syst	4c1 System stwierdza, że podany użytkownik nie posiada uprawnień do żadnych zasobów o czym go informuje, co kończy PU.
3 Użytkownik wprowadza żądane informacje i zatwierdza je.	
4 System stwierdza, że wprowadzono odpowiednie dane, weryfikuje tożsamość użytkownika oraz dokonuje autoryzacji.	
5 System daje użytkownikowi dostęp do odpowiadających mu zasobów.	



Dobre przypadki użycia

- 01** Używają interdyscyplinarnego języka
- 02** Opisują co system będzie robił
- 03** Zapisują podejmowane decyzje
- 04** Pozwalają weryfikować kompletność
- 05** Dostarczają kontekstu dla wymagań

Dobre przypadki użycia



SĄ	NIE SĄ
Tekst	Diagramami PU w UMLu
Brak GUI	Opisem GUI
Brak formatu danych	Opisem formatu danych
3 - 9 kroków w głównym scenariuszu	Wielostronicowym scenariuszem
Łatwo je zrozumieć	Nieczytelne
Przedstawiają potrzeby użytkownika	Opisują cechy programu
Zapisują podjęte decyzje	Opisem dziedziny biznesowej

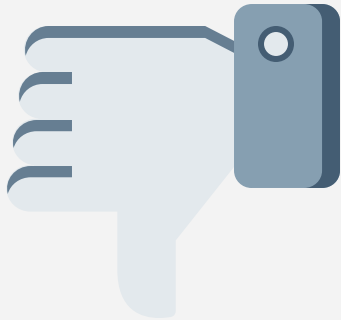
PU wady i zalety



Zapisują wymagania funkcjonalne



Stanowią szkielet dla wymagań
niefunkcjonalnych oraz harmonogramu



Zapisują **wyłącznie** wymagania funkcjonalne



Projektowanie nie ogranicza się do samych PU

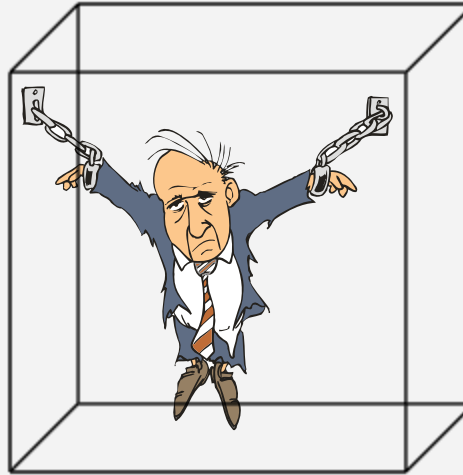
Przypadek użycia

Projektowany System (PS)

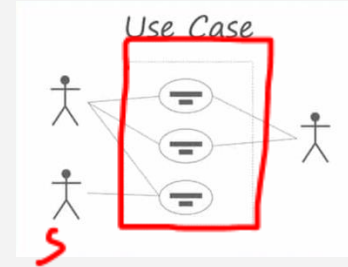


Główny Aktor

osoba lub system
chce wykorzystać PS
w określonym celu

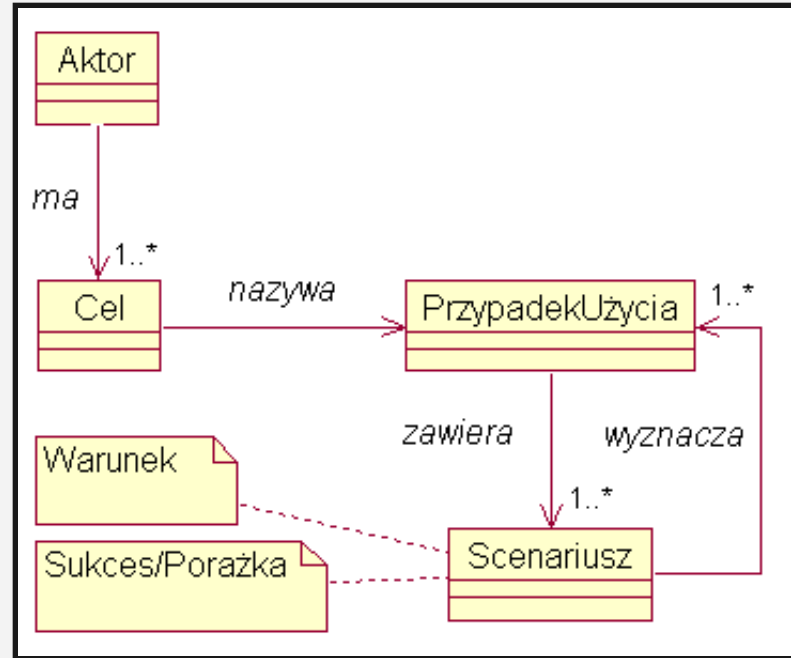
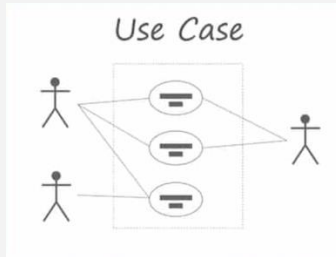


**Aktor
Drugoplanowy**
inny system
realizujący cel dla PS



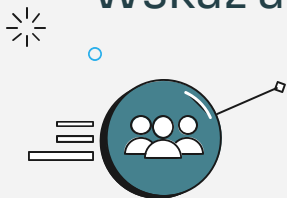
Przypadek użycia

- ✓ Aktor ma cele
- ✓ Cele nazywają PU
- ✓ Przypadki mają scenariusze nazywające podprzypadki



Jak pisać PU?

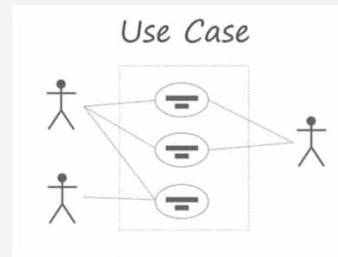
Wskaż aktorów i ich cele



- ✓ Aktor to coś/ktoś, co/kto wchodzi w interakcję z naszym systemem
- ✓ Czego potrzebuje od systemu aktor?

Napisz krótko

- ✓ Pomyślny scenariusz
- ✓ Wychwyc zamierzenia oraz odpowiedzialność wszystkich aktorów
 - ✓ Jak wymieniają między sobą informacje?
 - ✓ Ponumeruj linie



Jak pisać PU?

Wypisz warunki zakończone niepowodzeniem (porażki) jako rozszerzenia

- ✓ Zazwyczaj każdy krok może zakończyć się niepowodzeniem

Opisz porażki do momentu zakończenia lub powrotu do głównego scenariusza

Zwróć uwagę na zmieniające się dane

- ✓ Różne rodzaje dostawy
- ✓ Wskazują sytuacje, które muszą by obsługiwane przez PU niższego poziomu

Typowe wpadki

Register for Courses



1. Display a blank schedule.
2. Display a list of all classes in the following way:
The left window lists all the courses in the system in alphabetical order.
The lower window displays the times the highlighted course is available.
The third window shows all the courses currently in the schedule.
3. Do
4. Student clicks on a course.
5. Update the lower window to show the times the course is available.
6. Student clicks on a course time and then on the “Add Course” button.
7. Check if the Student has the necessary prerequisites and that the course offering is open.
8. If the course is open and the Student has the necessary prerequisites, add the Student to the course. Display the updated schedule showing the new course. If no, put up a message, “You are missing the prerequisites. Choose another course.”
9. Mark the course offering as “enrolled” in the schedule.
10. End do when the Student clicks on “Save Schedule.”
11. Save the schedule and return to the main selection screen.

Typowe wpadki

Register for Courses



1. ?Display a blank schedule.
2. ?Display a list of all classes **in the following way**:
The left window lists all the courses in the system in alphabetical order.
The lower window displays the times the highlighted course is available.
The third window shows all the courses currently in the schedule.
3. ?Do
4. Student **clicks** on a course.
5. ?Update **the lower window** to show the times the course is available.
6. Student **clicks** on a course time and then on **the “Add Course” button**.
7. ?Check if the Student has the necessary prerequisites and that the course offering is open.
8. ?If the course is open and the Student has the necessary prerequisites, add the Student to the course. **Display** the updated schedule showing the new course. If no, **put up a message**, “You are missing the prerequisites. Choose another course.”
9. ?Mark the course offering as “enrolled” in the schedule.
10. ?End do when the Student **clicks on “Save Schedule”**.
11. ?Save the schedule and **return to the main selection screen**.

Mogłoby być tak

Register for Courses



System: Course Enrollment System

Goal level: User Goal

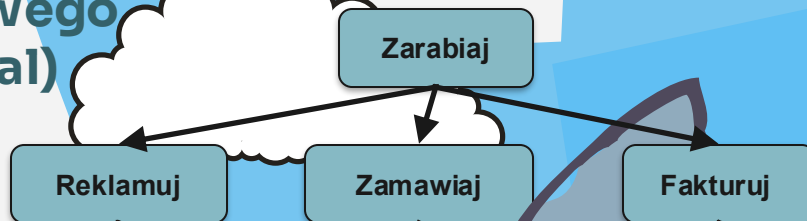
1. Student requests to construct a schedule.
2. The system prepares a blank schedule form.
3. The system gets available courses from the Course Catalog System.
4. Student selects up to 4 primary and 2 alternate course offerings.
5. For each course, the system verifies that the Student has the necessary prerequisites, adds the Student to the course, marking Student as “enrolled” for that course in the schedule.
6. When the Student indicates the schedule is complete, the system saves it.

Extensions:

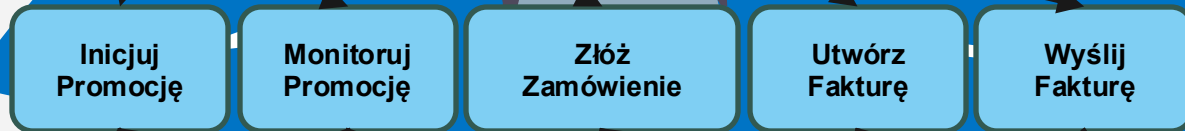
- 1a. Student already has a schedule: System brings up the current version of the Student’s schedule for editing instead of creating a new one.
- 1b. Current semester is closed and next semester is not yet open: System lets Student look at existing schedules, but not create new ones.
- 3a. Course Catalog System does not respond: The system notifies the Student and the use case ends.
- 5a. Course full or Student has not fulfilled all prerequisites: System disables selection of that course and notifies the Student.

Poziomy PU

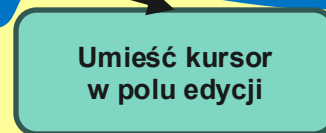
Celu biznesowego
(Summary Goal)



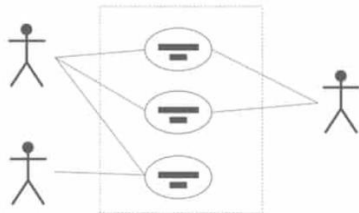
Użytkownika
(User Goal)



Podfunkcji



Use Case



Przypadki użycia

⚙️ Alistair Cockburn <https://alistair.cockburn.us>

⚙️ A. Cockburn, Jak pisać efektywne przypadki użycia

⚙️ S. Adolph, P. Bramble, A. Cockburn, A. Pols: Patterns for Effective Use Cases



**Robert C. Martin
(Uncle Bob)**

Author of Clean Code and many influential books, Co-author of the Agile Manifesto, **USA**



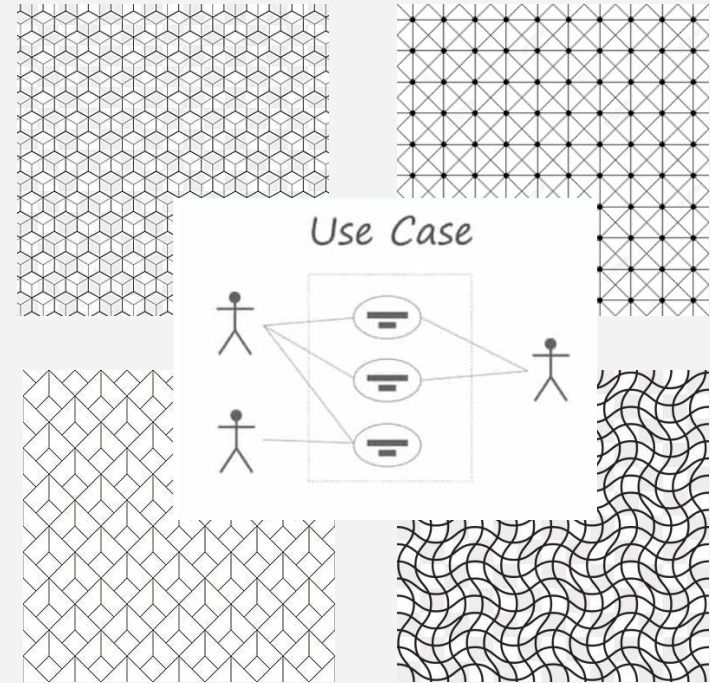
Wzorce dla PU

01 Pojedynczy PU

02 Zbiór PU

03 Proces tworzenia PU

04 Zespół autorów



Wzorce dla PU

01 Pojedynczy PU

02 Zbiór PU

03 Proces tworzenia PU

04 Zespół autorów

- ✓ Nazwa to fraza czasownikowa, akcja: GENERUJ RAPORT, RAPORT
- ✓ Scenariusz i rozszerzenia każdy po 3-9 kroków
- ✓ Obojętność technologiczna
 - ✓ Technologia jest zmienna
 - ✓ Niepotrzebne ograniczenia
 - ✓ GUI wprowadza niepotrzebne szczegóły
 - ✓ Klient nie rozumie terminów technicznych
 - ✗ ~~Student zaznacza checkbox~~

Wzorce dla PU

01 Pojedynczy PU

02 Zbiór PU

03 Proces tworzenia PU

04 Zespół autorów

- ✓ Hierarchia
- ✓ Można rozwijać lub zwijać w celu pokazania/ukrycia szczegółów

Wzorce dla PU

01 Pojedynczy PU

02 Zbiór PU

03 Proces tworzenia PU

04 Zespół autorów

✓ Pisz najpierw wszerz, potem w głąb

1. Lista aktorów
2. Nazwy PU
3. Główne scenariusze
4. Rozszerzenia

Wzorce dla PU

01 Pojedynczy PU

02 Zbiór PU

03 Proces tworzenia PU

04 Zespół autorów

✓ Mały

- ✓ 2-3 osoby (duży system to kilka zespołów)
- ✓ Więcej recenzentów

✓ Zrównoważony

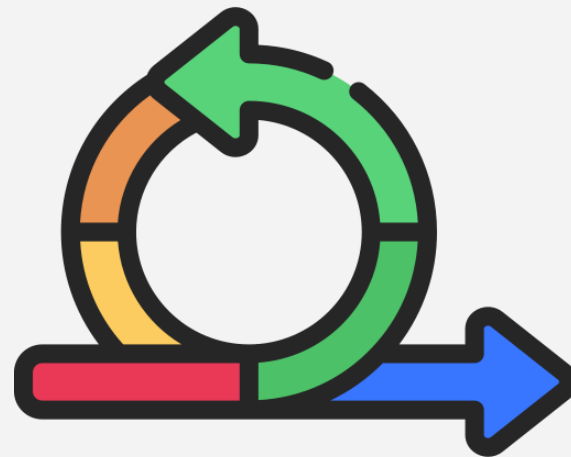
- ✓ Zróżnicowany (analityk + klient)
- ✓ Synergia!

• Zwinna specyfikacja wymagań •

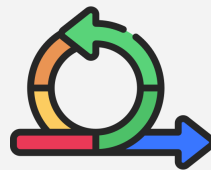
01 Karty wymagań (User Stories)

02 Szkice ekranów

03 Testy akceptacyjne



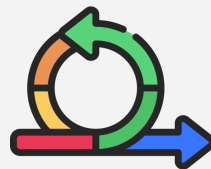
• Zwinna specyfikacja wymagań •



01 Karty wymagań (User Stories)

Jako [osoba odgrywająca daną rolę] **chciałbym móc**
[wykonać jakąś czynność] **aby** [osiągnąć jakiś cel].

Zwinna specyfikacja wymagań



01 Karty wymagań (User Stories)

UTWÓRZ KONTO

OPIS

Jako administrator chciałbym móc utworzyć konto pracownikowi dydaktycznemu, aby umożliwić mu dostęp do systemu.

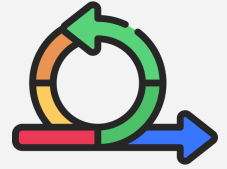
Po wypełnieniu wymaganych pól, system automatycznie generuje hasło oraz wysyła nowemu użytkownikowi wiadomość e-mail z danymi dostępowymi.

PRIORYTET

Krytyczne

OSZACOWANIE

6h



Zwinna specyfikacja wymagań

02 Szkice ekranów

A Web Page

http://

WFAIS Taskzilla

Jan Nowak (Administrator) [Zmien hasło](#) [Wylóżaj](#)

Pracownicy Semestry

Tytuł	Imię	Nazwisko	Login	Email	Status
dr	Katarzyna	Grzesiak-Kopec	K6	katarzyna.grzesiak-kopec@uj.edu.pl	aktywny
dr	Jan	Kowalski	j.kowalski	kowalski@uj.edu.pl	zablokowany
mgr	Mariah	Maciachlan	maciachlan	maciachlan@gmail.com	aktywny

Utwórz nowe konto

Tytuł:

Imię: *

Nazwisko: *

Login: *

E-mail: *

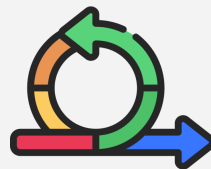
Telefon:

Pokój:

Zakład:

Konsultacje:

Zwinna specyfikacja wymagań

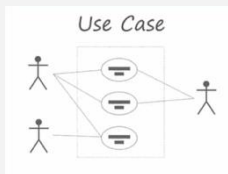


03 Testy akceptacyjne

UTWÓRZ KONTO testy akceptacyjne

- 1 Administrator wypełnił prawidłowo wszystkie wymagane pola, wprowadził również unikalną wartość identyfikatora użytkownika – w nowo otwartym oknie system informuje o pomyślnym utworzeniu konta i wysłaniu wiadomości e-mail
- 2 Administrator wypełnił wszystkie wymagane pola, jednak w systemie figuruje już użytkownik o takim samym identyfikatorze – system wyświetla na formularzu komunikat ostrzegawczy o tym, że pole „login” musi zawierać unikalną wartość.
- 3 Administrator nie wypełnił wymaganych pól (oznaczonych *) – system wyświetla na formularzu informację o niekompletnych danych wraz z prośbą o ich uzupełnienie.

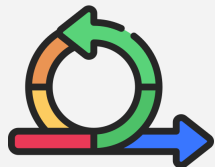
PRZYPADKI UŻYCIA



- ✓ Opisują proces jako sekwencję kroków
- ✓ Poziom szczegółowości pozwala na rozważanie ich pojedynczo

vs

KARTY WYMAGAŃ



- ✓ Informują jedynie o pewnej funkcjonalności
- ✓ Do jednoznacznej interpretacji trzeba dołączyć do nich szkice ekranów i testy akceptacyjne

UC vs US

... DON'T SUBDIVIDE...

A **user story** cut in half makes two smaller user stories, as a flatworm cut in half makes two smaller flatworms, but

a **use case** cut in half doesn't make two small UCs, as a horse cut in half doesn't make two small horses.

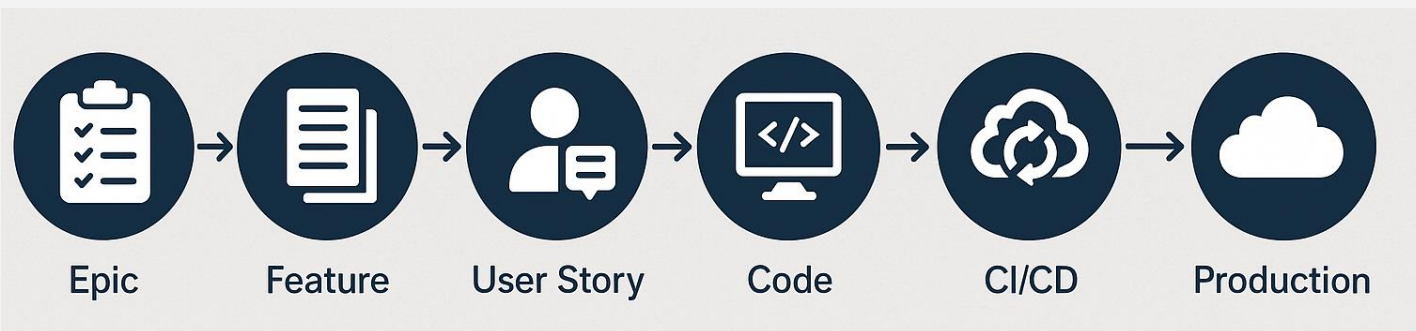
Alistair Cockburn



Agile & DevOps

W Agile i DevOps wymagania nie są statycznym dokumentem, lecz ciągle ewoluującym backlogiem.

Każdy element (epik, funkcja, historia użytkownika) przechodzi pełen cykl — od definicji do wdrożenia w pipeline CI/CD.



Agile & DevOps

Poziom ▼	Opis ▼	Horyzont czasowy ▼
Epic	Duży cel biznesowy lub obszar funkcjonalny (np. „System płatności”)	miesiące
Feature	Zbiór spójnych funkcji realizujących część epiku	tygodnie
User Story	Konkretna potrzeba użytkownika: „ <i>Jako klient chcę zapłacić kartą, aby sfinalizować zakup</i> ”	dni
Task / Sub-task	Techniczne zadania niezbędne do realizacji story	godziny

📌 Każdy poziom backlogu może być śledzony w narzędziach takich jak Jira, Azure DevOps, Trello.

Definition of Ready (DoR)

DoR – user story gotowe do rozpoczęcia prac, gdy opisano:
cel, kryteria akceptacji,
zależności i priorytet

Definition of Done (DoD)

DoD – story jest ukończone, gdy:
spełnia kryteria akceptacji,
przeszło testy (manualne i automatyczne),
wdrożono je w środowisku CI/CD,
zostało zrecenzowane i
udokumentowane

Agile & DevOps : wymagania niefunkcjonalne

Wymagania jakościowe (wydajność, bezpieczeństwo, UX) są włączane jako:

- ✓ Kryteria akceptacji w US
- ✓ Definition of Done (testy bezpieczeństwa przeszły 100%)
- ✓ Elementy backlogu technicznego (np. „poprawa czasu ładowania strony < 2s”).

Traceability – od wymagania do wdrożenia:

User Story → Commit → Test → Build → Deployment

Narzędzia wspierające: Jira + Git + Jenkins / GitLab CI + TestRail / Xray

Narzędzia wspierające inżynierię wymagań

Backlog & wymagania funkcjonalne

Narzędzie	Główne zastosowanie	Wyróżniki
Jira Software	Zarządzanie backlogiem, epikami i user stories	Integracja z CI/CD i testami (Xray, Zephyr)
Azure DevOps	Planowanie, backlogi, traceability	Silna integracja z Git, pipelines i testami
Trello	Wizualne tablice Kanban dla małych zespołów	Prostota, integracja z automatyzacją
Notion	Dokumentacja, backlogi, taski w jednym miejscu	Elastyczne szablony i współpraca w czasie rzeczywistym
Miro	Warsztaty, mapy wymagań, story mapping	Współtworzenie wizualne, idealne dla pracy zdalnej



Narzędzia wspierające inżynierię wymagań

Dokumentacja & wiedza projektowa

Narzędzie	Zastosowanie	Integracje
Confluence	Centralna baza wiedzy, dokumentacja SRS i user stories	Bezpośrednio z Jira
Notion	Elastyczna, współdzielona dokumentacja projektowa	Integracje z GitHub, Slack, Figmą
GitHub Projects / Wiki	Dokumentacja techniczna w repozytorium kodu	Traceability z commitami i pull requestami

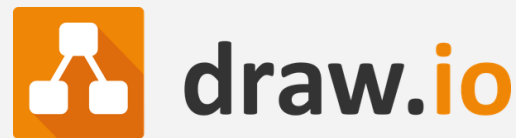


„Żywa dokumentacja” (living documentation) powstaje automatycznie z repozytorium i testów

Narzędzia wspierające inżynierię wymagań

Wizualizacja modeli & architektury

Narzędzie	Typ diagramów	Zastosowanie
PlantUML	UML, C4, sekwencje, przypadki użycia	Generowanie diagramów z plików tekstowych (.puml)
Structurizr	Modele architektury C4	Zintegrowane z repozytorium kodu
Lucidchart / Draw.io (Diagrams.net)	Diagramy procesów, przepływy danych, mapy kontekstu	Praca zespołowa, integracje z Atlassian i GitHub



„Żywa dokumentacja” (living documentation) powstaje automatycznie z repozytorium i testów

Narzędzia wspierające inżynierię wymagań

Testy akceptacyjne & walidacja wymagań

Narzędzie / Technika	Opis	Cel
BDD (Behavior Driven Development)	Opisywanie zachowań systemu w języku naturalnym	Łączy wymagania z testami
Gherkin	Format plików .feature (Given-When-Then)	Formalizacja scenariuszy użytkownika
Cucumber / SpecFlow	Uruchamianie testów akceptacyjnych BDD	Integracja z CI/CD (np. Jenkins, GitLab CI)



Feature: Guess the word

The first example has two steps

Scenario: Maker starts a game

When the Maker starts a game

Then the Maker waits for a Breaker to join

The second example has three steps

Scenario: Breaker joins a game

Given the Maker has started a game with the word "silky"

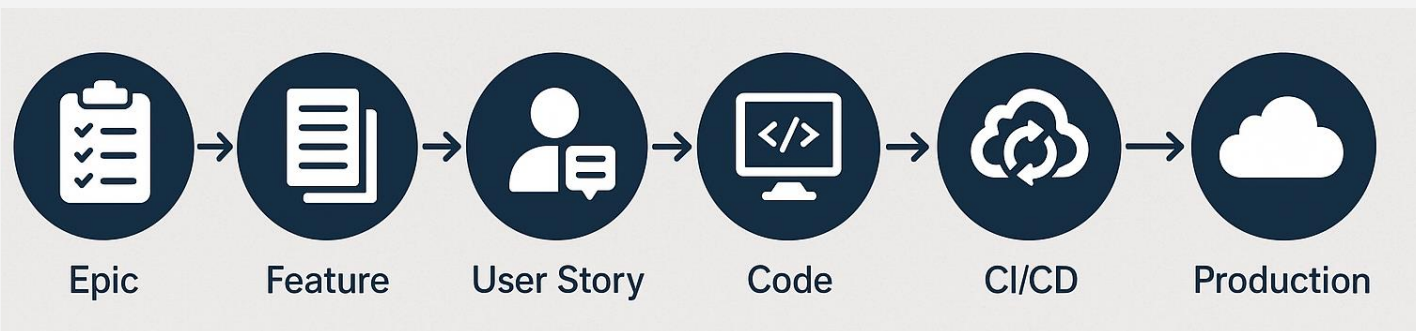
When the Breaker joins the Maker's game

Then the Breaker must guess a word with 5 characters

💬 Dzięki BDD wymagania są częścią automatycznych testów akceptacyjnych – zawsze aktualnych.

• Agile & DevOps

Współczesna inżynieria wymagań to ekosystem narzędzi połączonych w jednym procesie – od backlogu i modeli po testy i wdrożenie w DevOps.





Koniec!